

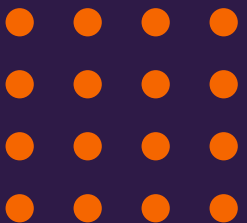


Support Vector Machines

Tim Ransom



College of
**ENGINEERING, COMPUTING
AND APPLIED SCIENCES**



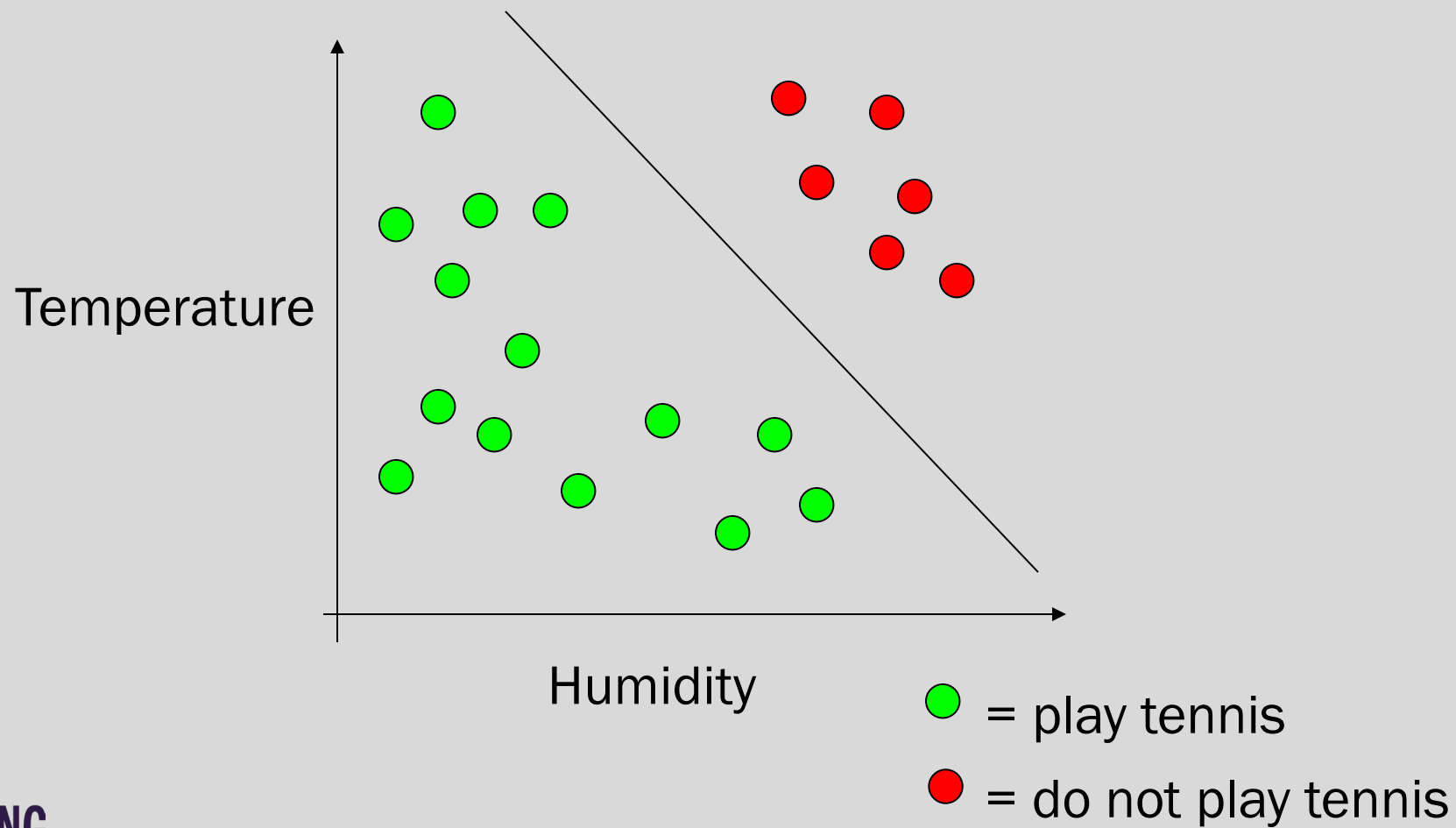


Main Ideas

- Max-Margin Classifier (Separating hyperplane)
 - Formalize notion of the best linear separator
- Lagrangian Multipliers
 - Way to convert a constrained optimization problem to one that is easier to solve
- Kernels
 - Projecting data into higher-dimensional space makes it linearly separable
- Complexity
 - Depends only on the number of training examples, not on dimensionality of the kernel space!

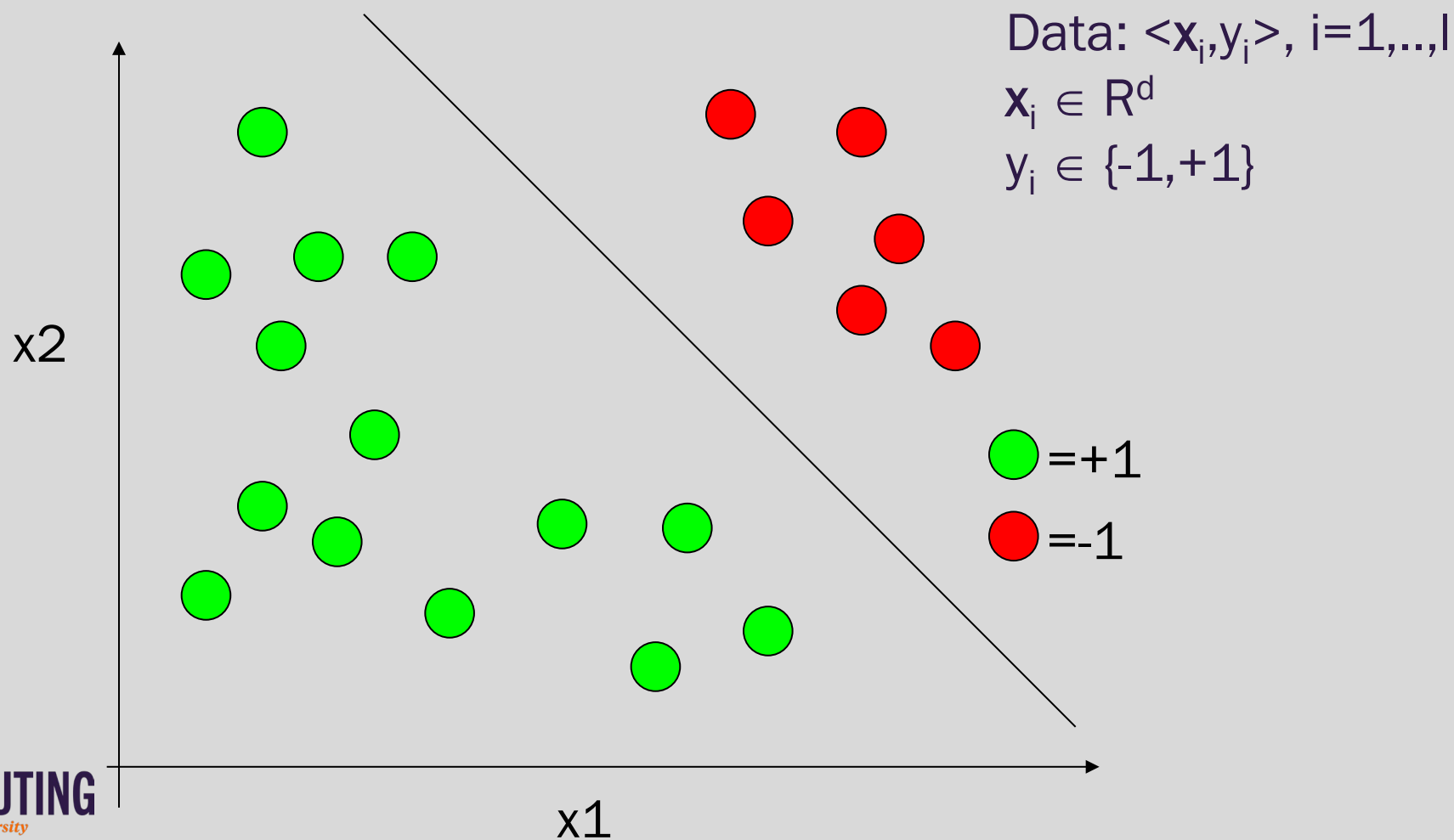


Tennis example





Linear Support Vector Machines



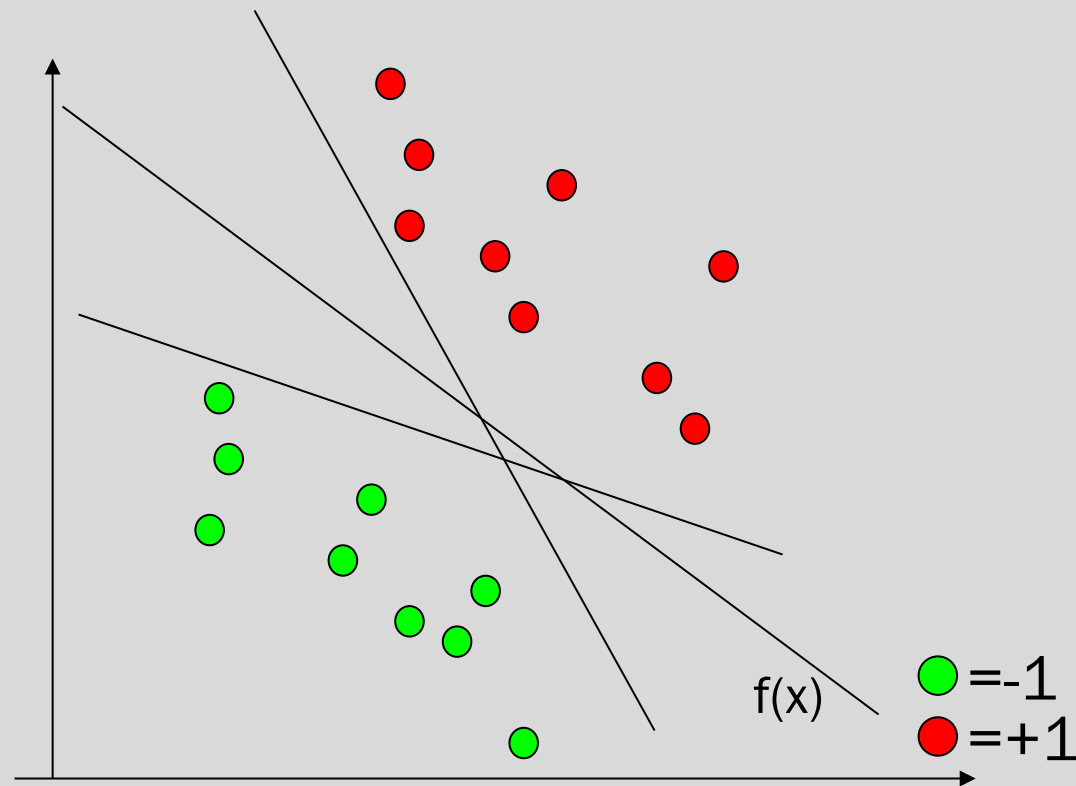


Linear SVM 2

All hyperplanes in R^d are parameterize by a vector ($\mathbf{w}$) and a constant b .

Can be expressed as $\mathbf{w} \cdot \mathbf{x} + b = 0$
(remember the equation for a hyperplane from algebra!)

Our aim is to find such a hyperplane $f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$, that correctly classifies our data.



Data: $\langle \mathbf{x}_i, y_i \rangle, i=1, \dots, l$

$\mathbf{x}_i \in R^d$

$y_i \in \{-1, +1\}$



Definitions

Define the hyperplane H such that:

$$\mathbf{x}_i \cdot \mathbf{w} + b \geq +1 \text{ when } y_i = +1$$

$$\mathbf{x}_i \cdot \mathbf{w} + b \leq -1 \text{ when } y_i = -1$$

H_1 and H_2 are the planes:

$$H_1: \mathbf{x}_i \cdot \mathbf{w} + b = +1$$

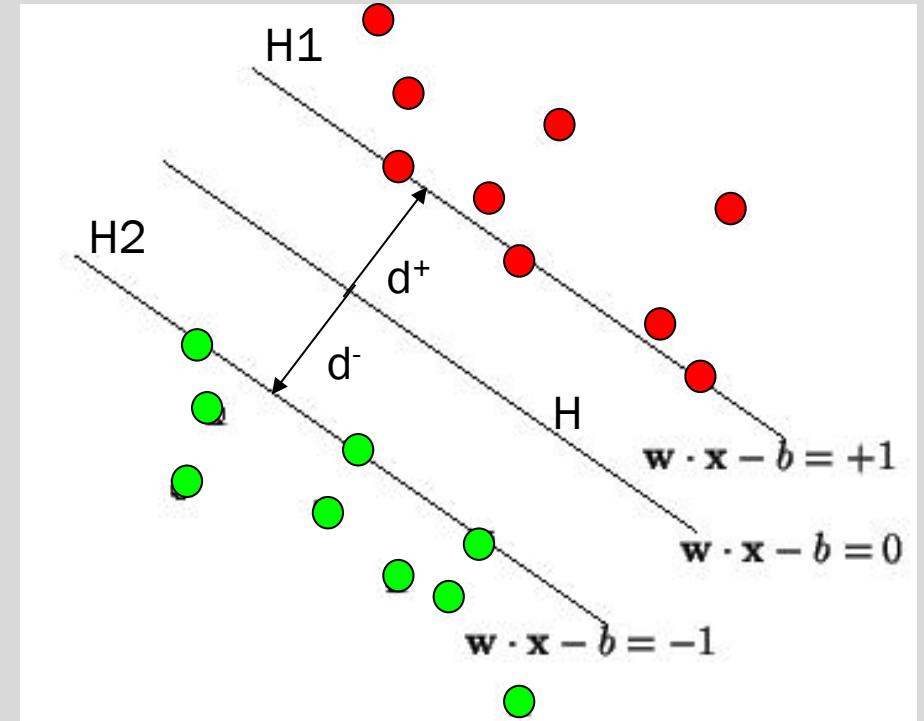
$$H_2: \mathbf{x}_i \cdot \mathbf{w} + b = -1$$

The points on the planes H_1 and H_2 are the Support Vectors

d^+ = the shortest distance to the closest positive point

d^- = the shortest distance to the closest negative point

The margin of a separating hyperplane is $d^+ + d^-$.





Maximizing the Margin

We want a classifier with as big of margin as possible

Recall the distance from a point (x_0, y_0) to a line:

$$Ax + By + c = 0 \text{ is } \frac{|Ax_0 + By_0 + c|}{\sqrt{A^2 + B^2}}$$

The distance between H and H_1 is: $\frac{|w \cdot x + b|}{||w||} = \frac{1}{||w||}$

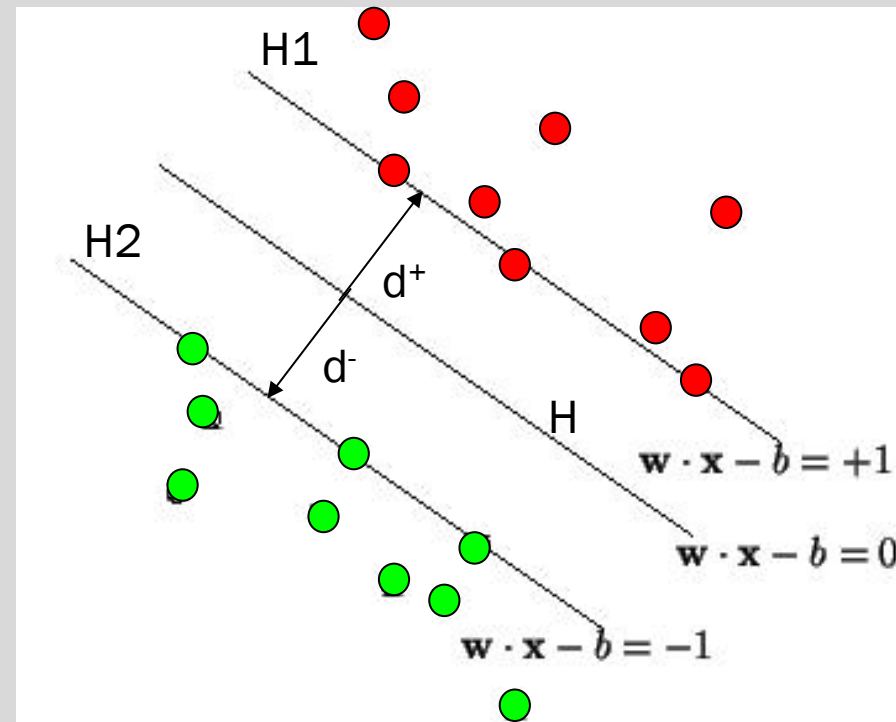
The distance between H_1 and H_2 is: $\frac{2}{||w||}$

In order to maximize the margin, we need to minimize $||w||$. With the condition that there are no datapoints between H_1 and H_2 :

$$x_i \cdot w + b \geq +1 \text{ when } y_i = +1$$

$$x_i \cdot w + b \leq -1 \text{ when } y_i = -1$$

Can be combined into $y_i(x_i \cdot w) \geq 1$





Constrained Optimization Problem

Minimize $\|\mathbf{w}\| = \langle \mathbf{w} \cdot \mathbf{w} \rangle$ subject to $y_i(\langle \mathbf{x}_i \cdot \mathbf{w} \rangle + b) \geq 1$ for all i

Lagrangian method: maximize $\inf_{\mathbf{w}} L(\mathbf{w}, b, \alpha)$, where

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i [(y_i(\mathbf{x}_i \cdot \mathbf{w}) + b) - 1]$$

At the extremum, the partial derivative of L with respect to both $\mathbf{w}$ and b must be 0. Taking the derivatives, setting them to 0, substituting back into L , and simplifying yields:

$$\text{Maximize } \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle$$

$$\text{subject to } \sum_i y_i \alpha_i = 0 \text{ and } \alpha_i \geq 0$$



Quadratic Programming

- Why is this reformulation a good thing?
- The problem

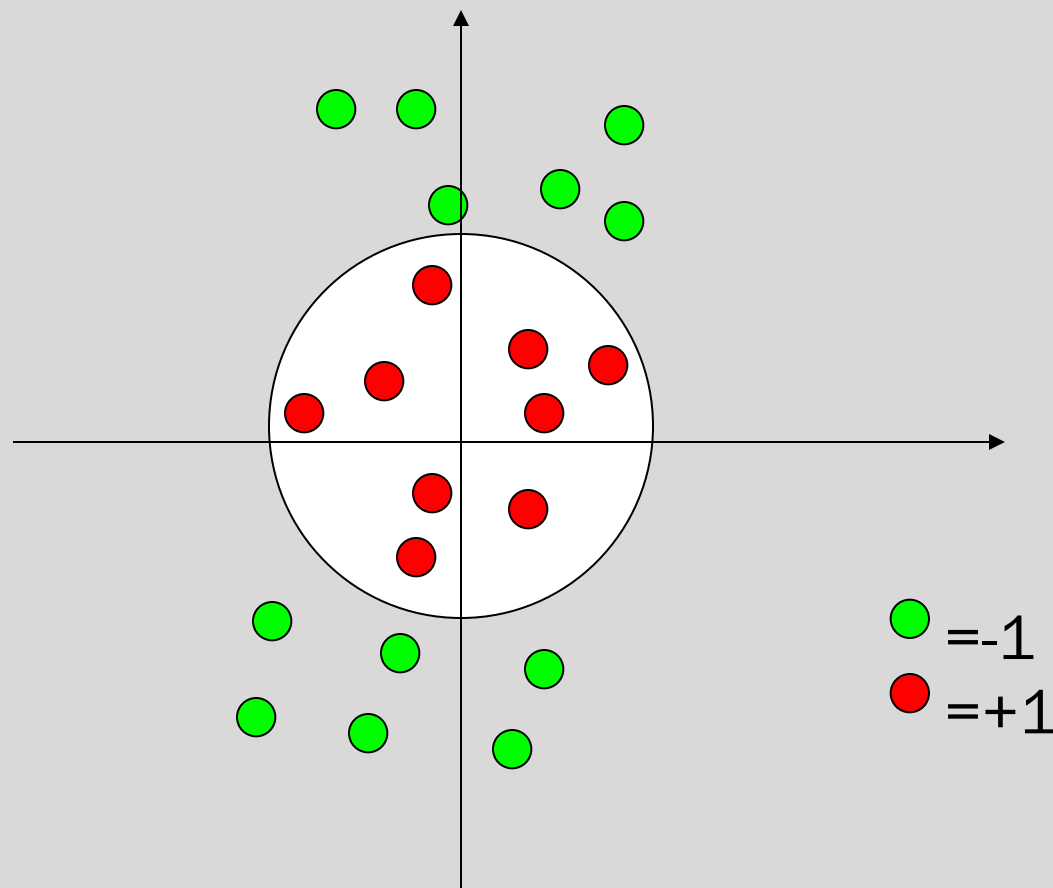
$$\begin{aligned} &\text{Maximize } \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle \\ &\text{subject to } \sum_i y_i \alpha_i = 0 \quad \text{and} \quad \alpha_i \geq 0 \end{aligned}$$

is an instance of what is called a positive, semi-definite programming problem

- For a fixed real-number accuracy, can be solved in $O(n \log n)$ time = $O(|D|^2 \log |D|^2)$



Problems with linear SVM

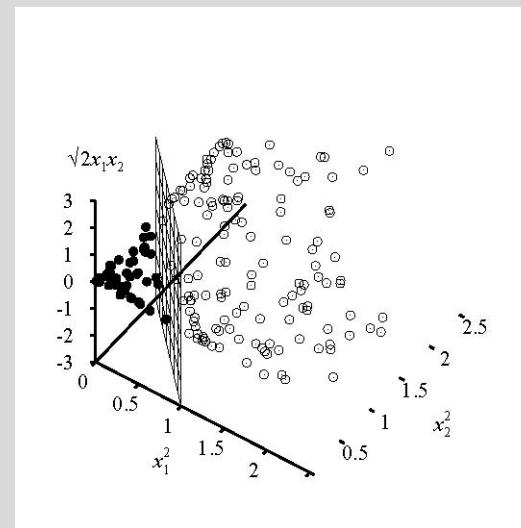


What if the decision function is not a linear?



Kernel Trick

Data points are linearly separable
in the space $(x_1^2, x_2^2, \sqrt{2}x_1x_2)$

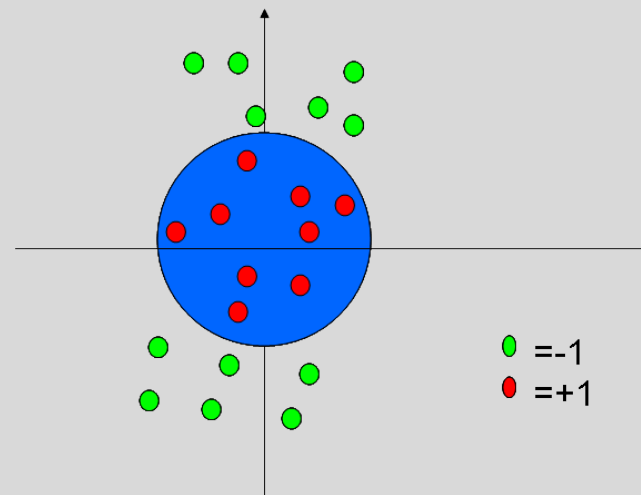


We want to maximize $\sum_i \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle F(\mathbf{x}_i) \cdot F(\mathbf{x}_j) \rangle$

Define $K(\mathbf{x}_i, \mathbf{x}_j) = \langle F(\mathbf{x}_i) \cdot F(\mathbf{x}_j) \rangle$

Cool thing: K is often easy to compute directly! Here,

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle^2$$





Other Kernels

The polynomial kernel

$K(x_i, x_j) = (x_i \bullet x_j + 1)^p$, where p is a tunable parameter.

Evaluating K only require one addition and one exponentiation more than the original dot product.

Gaussian kernels (also called radius basis functions)

$$K(x_i, x_j) = \exp(||x_i - x_j ||^2/2\sigma^2)$$

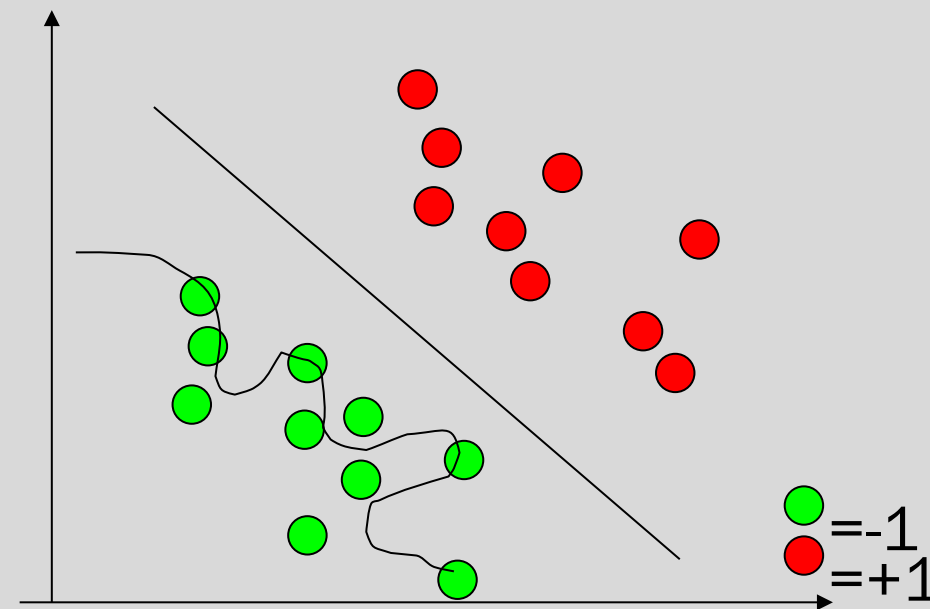


Overtraining/overfitting

A well-known problem with machine learning methods is overtraining.

This means that we have learned the training data very well, but we can not classify unseen examples correctly.

An example: A botanist thinks they know all trees. Every time he sees a new tree, he claims it is not a tree.





Overtraining/overfitting 2

A measure of the risk of overtraining with SVM (there are also other measures).

It can be shown that: The portion, n , of unseen data that will be misclassified is bounded by:

$$n \leq \text{Number of support vectors} / \text{number of training examples}$$

Ockham's razor principle: Simpler system are better than more complex ones.

In SVM case: fewer support vectors mean a simpler representation of the hyperplane.

Example: Understanding a certain cancer if it can be described by one gene is easier than if we must describe it with 5000.

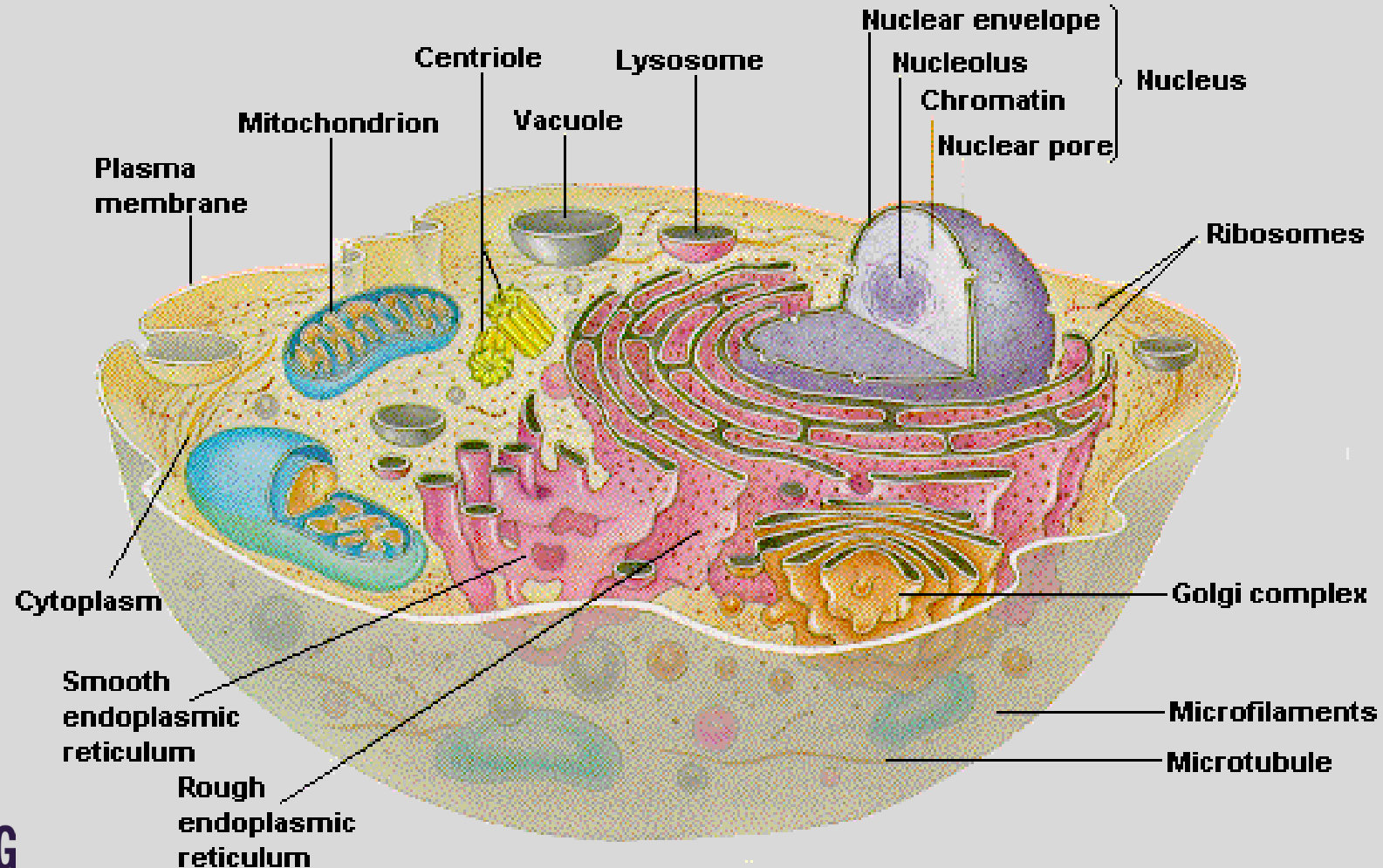


A practical example, protein localization

- Proteins are synthesized in the cytosol.
- Transported into different subcellular locations where they carry out their functions.
- Aim: To predict in what location a certain protein will end up!!!



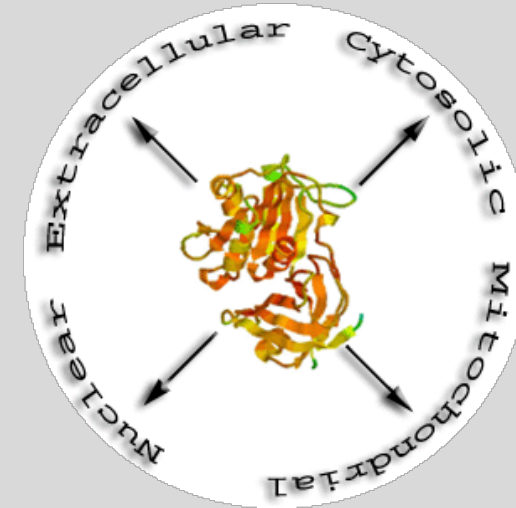
Subcellular Locations





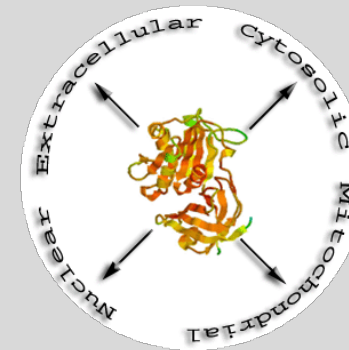
Method

- Hypothesis: The amino acid composition of proteins from different compartments should differ.
- Extract proteins with know subcellular location from SWISSPROT.
- Calculate the amino acid composition of the proteins.
- Try to differentiate between: cytosol, extracellular, mitochondria and nuclear by using SVM



Input encoding

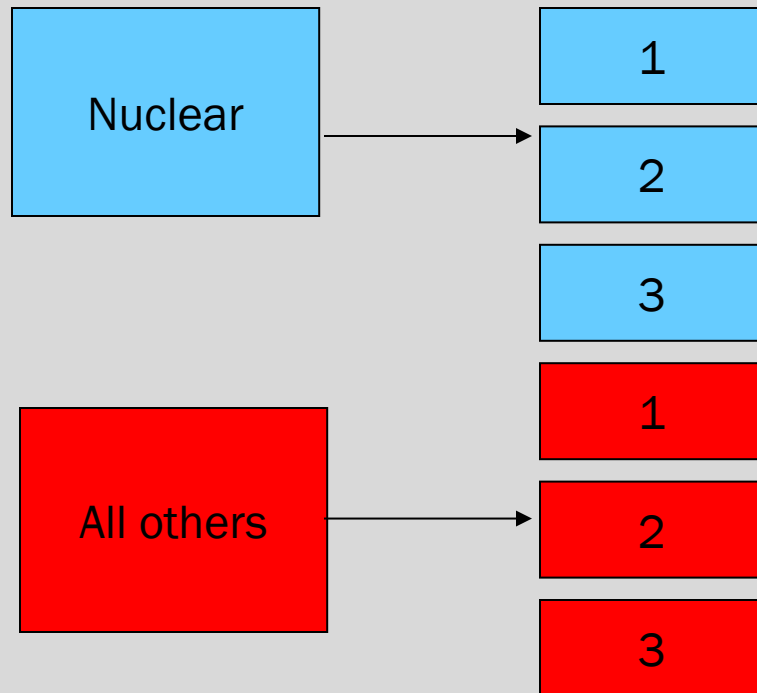
- Prediction of nuclear proteins:
- Label the known nuclear proteins as +1 and all others as -1.
- The input vector x_i represents the amino acid composition.
- Eg $x_i = (4.2, 6.7, 12, \dots, 0.5)$
(A, C, D, ..., Y)



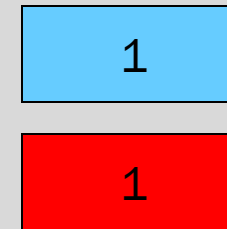


Cross-validation

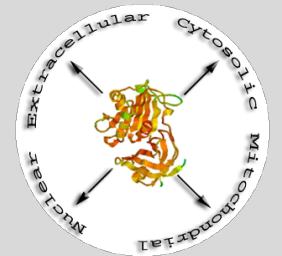
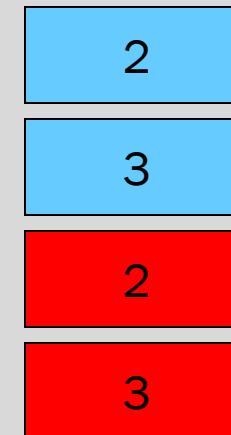
Cross validation: Split the data into n sets, train on $n-1$ set, test on the set left out of training.

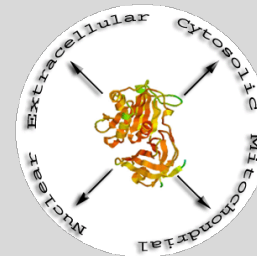


Test set

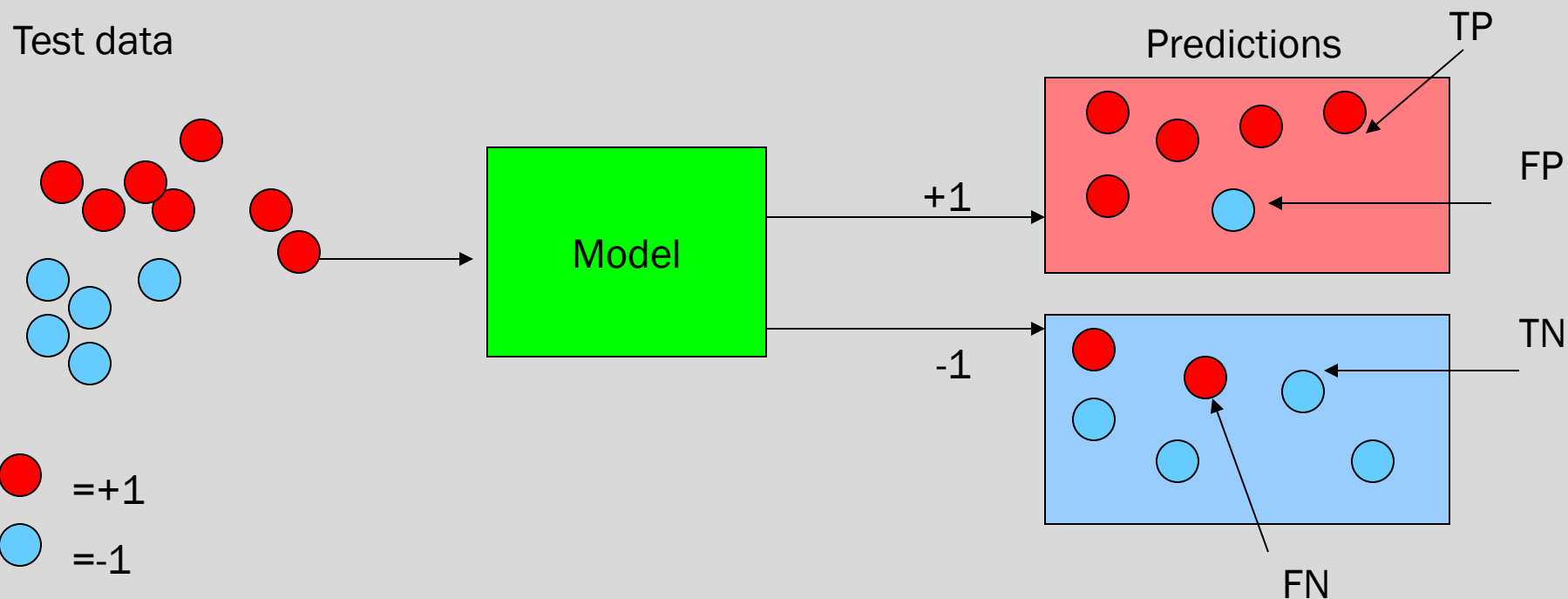


Training set





Performance measurements



SP = $TP / (TP + FP)$, the fraction of predicted +1 that actually are +1.

SE = $TP / (TP + FN)$, the fraction of the +1 that actually are predicted as +1.

In this case: $SP = 5 / (5 + 1) = 0.83$

$SE = 5 / (5 + 2) = 0.71$



A Cautionary Example

Image classification of tanks. Autofire when an enemy tank is spotted.

Input data: Photos of own and enemy tanks.

Worked really good with the training set used. In reality it failed completely.

Reason: All enemy tank photos taken in the morning. All own tanks in dawn.

The classifier could recognize dusk from dawn!!!!





References

<http://www.kernel-machines.org/>

<http://www.support-vector.net/>

AN INTRODUCTION TO SUPPORT VECTOR MACHINES

(and other kernel-based learning methods)

N. Cristianini and J. Shawe-Taylor

Cambridge University Press

2000 ISBN: 0 521 78019 5

Papers by Vapnik

C.J.C. Burges: A tutorial on Support Vector Machines. Data Mining and Knowledge Discovery 2:121-167, 1998.